

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Análise de uma Sessão Multimídia em Tempo Real Utilizando o StreamYard

Projeto Final - Sistemas Multimídia

Disciplina: SMU029009 - SISTEMAS MULTIMÍDIA (2026 .1 - T01)

Professora Ana Luiza Scharf

Alunos: Júlia Espindola Steinbach e Wagner Flores dos Santos

Julho de 2026

Sumário

1. Introdução	3
2. Descrição do Experimento	3
3. Sinalização	4
3.1. Exemplo de sinalização observada	6
4. Negociação de Mídia	7
4.1. Captura local de áudio e vídeo	7
4.2. Áudio	8
4.3. Vídeo	10
5. Escolha de Caminho	12
6. Transporte de Mídia	14
7. Estatísticas da Sessão	16
8. Qualidade de Serviço	17
9. Segurança	17
10. Conclusão	18
11. Referências	19

1. Introdução

O presente trabalho tem como objetivo analisar o funcionamento de uma aplicação multimídia em tempo real, observando os principais mecanismos utilizados para o estabelecimento, transporte e controle de uma sessão audiovisual.

Como plataforma de estudo foi escolhido o StreamYard, uma ferramenta baseada em WebRTC amplamente utilizada para transmissões ao vivo, entrevistas remotas e produção de conteúdo audiovisual via navegador.

O experimento consistiu na criação de uma sessão privada contendo dois participantes, utilizando áudio e vídeo bidirecionais durante aproximadamente três minutos. Durante a execução foram realizadas capturas através do Chrome WebRTC Internals e do Wireshark, permitindo observar os protocolos envolvidos na sinalização, negociação de mídia, escolha de caminho e transporte dos fluxos multimídia.

2. Descrição do Experimento

O experimento foi realizado utilizando dois participantes conectados ao mesmo estúdio virtual do StreamYard.

- Participante 1: Wagner Santos (Host)
- Participante 2: Julia (Convidada)
- Aplicação: StreamYard
- Navegador: Google Chrome
- Tipo de mídia: Áudio e Vídeo
- Duração aproximada: 3 minutos

Durante a sessão foram coletadas informações através das ferramentas Chrome WebRTC Internals e Wireshark para posterior análise dos protocolos envolvidos.

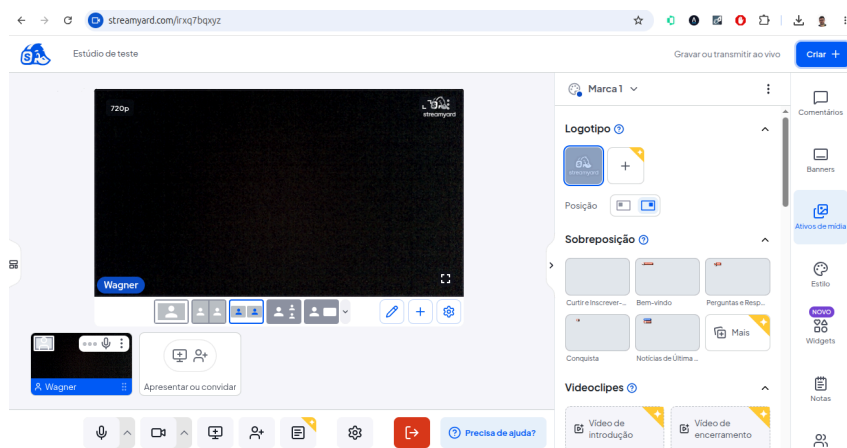


Figure 1: Estúdio de teste do StreamYard

3. Sinalização

O StreamYard utiliza WebRTC para o estabelecimento da comunicação multimídia. Em aplicações WebRTC, a sinalização é responsável por permitir que os participantes troquem informações necessárias para iniciar a sessão, como descrições SDP, candidatos ICE e parâmetros de mídia.

No experimento, a sinalização ocorreu por meio da infraestrutura própria do StreamYard. Embora as mensagens internas da aplicação sejam proprietárias, o WebRTC Internals permitiu observar o resultado do processo de negociação, incluindo a criação das PeerConnections, a configuração dos servidores ICE e as estatísticas associadas à sessão.

O fluxo geral observado pode ser descrito da seguinte forma:

- criação da sessão no StreamYard;
- acesso aos dispositivos locais de áudio e vídeo;
- criação da PeerConnection;
- negociação de mídia por SDP;
- coleta e troca de candidatos ICE;
- seleção do par de candidatos;
- estabelecimento da sessão de mídia.

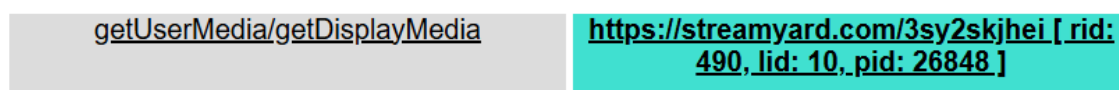


Figure 2: Captura do Chrome WebRTC Internals.

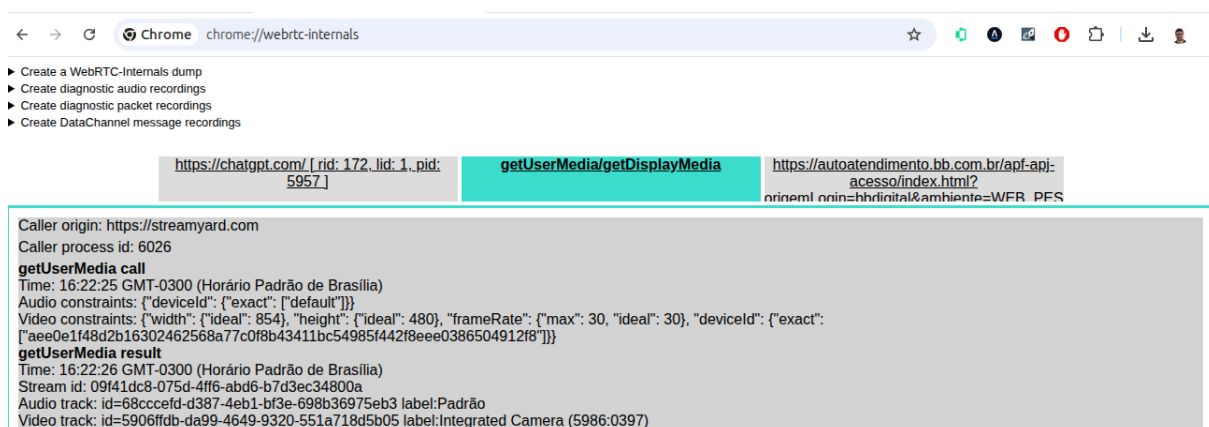


Figure 3: Captura do pedido de mídia no Chrome WebRTC Internals durante a sessão analisada.

As Figuras abaixo representam detalhes da sessão do StreamYard, selecionado na Figura 2, que aparece duplicado em sessão de quem cria a sala (host) e quem entra nela (convidado).

Time	Event
16/06/2026, 20:33:04	▶ transceiverAdded(index=0, kind=audio)
16/06/2026, 20:33:04	▶ transceiverAdded(index=1, kind=video)
16/06/2026, 20:33:04	▶ createOffer
16/06/2026, 20:33:04	onnegotiationneeded
16/06/2026, 20:33:04	▶ createOfferOnSuccess (type: "offer", 3 sections)
16/06/2026, 20:33:04	▶ setLocalDescription (type: "offer", 3 sections)
16/06/2026, 20:33:04	setLocalDescriptionOnSuccess
16/06/2026, 20:33:04	▶ onsignalingstatechange
16/06/2026, 20:33:04	▶ transceiverModified(index=0, kind=audio)
16/06/2026, 20:33:04	▶ transceiverModified(index=1, kind=video)
16/06/2026, 20:33:04	▶ onicegatheringstatechange
16/06/2026, 20:33:04	▶ onicecandidate

Figure 4: Captura do Chrome WebRTC Internals com SDP offer durante a criação da sala (host).

Time	Event
16/06/2026, 20:35:37	▶ setRemoteDescription (type: "offer", 3 sections)
16/06/2026, 20:35:37	setRemoteDescriptionOnSuccess
16/06/2026, 20:35:37	▶ onsignalingstatechange
16/06/2026, 20:35:37	▶ transceiverAdded(index=0, kind=video)
16/06/2026, 20:35:37	▶ transceiverAdded(index=1, kind=audio)
16/06/2026, 20:35:37	▶ createAnswer
16/06/2026, 20:35:37	▶ createAnswerOnSuccess (type: "answer", 3 sections)
16/06/2026, 20:35:37	▶ setLocalDescription (munged) (type: "answer", 3 sections)
16/06/2026, 20:35:37	setLocalDescriptionOnSuccess
16/06/2026, 20:35:37	▶ onsignalingstatechange
16/06/2026, 20:35:37	▶ transceiverModified(index=0, kind=video)
16/06/2026, 20:35:37	▶ transceiverModified(index=1, kind=audio)
16/06/2026, 20:35:37	▶ onicegatheringstatechange
16/06/2026, 20:35:37	▶ oniceconnectionstatechange
16/06/2026, 20:35:37	▶ onconnectionstatechange

Figure 5: Captura do Chrome WebRTC Internals com SDP answer na entrada do convidado.

O estabelecimento da sessão no StreamYard ocorre por meio da troca de descrições SDP no modelo Offer/Answer do WebRTC. O participante que cria a sala gera a SDP Offer, equivalente conceitualmente a uma mensagem SIP INVITE, contendo as mídias e parâmetros suportados. Quando um novo participante ingressa na sala, essa oferta é aplicada como descrição remota e respondida com uma SDP Answer, equivalente a uma resposta SIP 200

OK. A sessão é considerada estabelecida quando o estado de sinalização atinge o valor `stable`, indicando a conclusão da negociação.

```
https://streamyard.com/3sy2skjhei, [{"iceServers":[{"urls":["stun:turn-prod-1.streamyard.com:443"]}, {"urls":["turn:turn-prod-1.streamyard.com:443?transport=udp","turn:turn-prod-1.streamyard.com:443?transport=tcp"], "turns":turn-prod-1.streamyard.com:443?transport=tcp"}], "alwaysNegotiateDataChannels":false},  
ICE connection state: new => "checking" => "connected" => "completed" => "completed"  
Connection state: new => "connecting" => "connected" => "connected"  
Signaling state: new => "have-local-offer" => "stable" => "stable"
```

Figure 6: Captura do Chrome WebRTC Internals com logs de sinalização.

Os logs de sinalização demonstram o funcionamento completo do processo de negociação WebRTC. Inicialmente, o estado de sinalização evolui para `have-local-offer`, indicando que o usuário que criou a sala gerou um SDP Offer contendo os parâmetros de mídia e transporte. Em seguida, o estado retorna para `stable`, caracterizando o recebimento do SDP Answer pelo participante que ingressou na sala, concluindo a negociação. Paralelamente, observa-se a evolução do estado ICE de `new` até `completed`, confirmando a descoberta e seleção do melhor caminho de rede para transmissão de mídia.

O encerramento da sessão no WebRTC ocorre com o fechamento da `PeerConnection`, procedimento equivalente ao envio da mensagem SIP BYE. Após o encerramento, os estados de conexão e ICE transitam para `closed`, indicando a liberação dos recursos de mídia e transporte.

O diálogo no WebRTC é caracterizado pela existência de uma instância ativa de `PeerConnection` associada a um par de descrições SDP (Offer/Answer) e aos parâmetros ICE negociados. Enquanto a conexão permanece no estado `connected`, considera-se que o diálogo entre as entidades está ativo, de forma análoga à definição de diálogo no protocolo SIP.

3.1. Exemplo de sinalização observada

```
"origin": "https://streamyard.com",  
"request_type": "getUserMedia",  
"video": {  
  "width": {"ideal": 854},  
  "height": {"ideal": 480},  
  "frameRate": {"ideal": 30}  
}
```

O trecho acima demonstra uma etapa inicial da negociação WebRTC, na qual a aplicação solicita acesso aos dispositivos locais de áudio e vídeo antes da criação da `PeerConnection` e do intercâmbio das descrições SDP.

```
"candidateType": "host",  
"address": "192.168.15.4",  
"protocol": "udp"
```

O candidato acima representa o endereço local inicialmente ofertado ao mecanismo ICE para estabelecimento da conectividade.

```
▼ createOfferOnSuccess (type: "offer", 3 sections)
Copy description to clipboard
▼ v=0 (6 more lines)
o=- 5274383181070937964 2 IN IP4 127.0.0.1
s=-
t=0 0
a=group:BUNDLE 0 1
a=extmap-allow-mixed
a=msid-semantic: WMS b29eddce-0b4d-4d52-85d0-2b3d68cda1bd
▶ m=audio 9 UDP/TLS/RTP/SAVPF 111 63 9 0 8 13 110 126 (29 more lines) direction=sendonly mid=0
▶ m=video 9 UDP/TLS/RTP/SAVPF 96 97 103 104 107 108 109 114 115 116 117 118 39 40 45 46 98 99 100 101 119 120 49 50

16/06/2026, 20:33:04

▼ setLocalDescription (type: "offer", 3 sections)
Copy description to clipboard
▼ v=0 (6 more lines)
o=- 5274383181070937964 2 IN IP4 127.0.0.1
s=-
t=0 0
a=group:BUNDLE 0 1
a=extmap-allow-mixed
a=msid-semantic: WMS b29eddce-0b4d-4d52-85d0-2b3d68cda1bd
▶ m=audio 9 UDP/TLS/RTP/SAVPF 111 63 9 0 8 13 110 126 (29 more lines) direction=sendonly mid=0
▶ m=video 9 UDP/TLS/RTP/SAVPF 96 97 103 104 107 108 109 114 115 116 117 118 39 40 45 46 98 99 100 101 119 120 49 50

16/06/2026, 20:33:04
```

Figure 7: Captura do Chrome WebRTC Internals com detalhes dos logs de sinalização.

No WebRTC, o conceito de diálogo não é explicitado por identificadores formais como no SIP. O diálogo é representado implicitamente por uma instância ativa de `RTCPeerConnection`, estabelecida após a troca bem-sucedida de uma SDP Offer e uma SDP Answer. Esse diálogo é caracterizado como mostra acima, pelos identificadores da sessão SDP (campo `o=`), pelos identificadores de mídia (`mid`), pelos parâmetros ICE (`ufrag` e `pwd`) e pelo estado de sinalização `stable`.

4. Negociação de Mídia

A negociação de mídia foi realizada através do SDP, responsável por descrever as capacidades de áudio e vídeo disponíveis entre os participantes. A partir dos dumps exportados pelo WebRTC Internals, foi possível identificar os dispositivos utilizados, os codecs selecionados e os parâmetros efetivos da transmissão.

4.1. Captura local de áudio e vídeo

O StreamYard solicitou ao navegador acesso à câmera e ao microfone através da API `getUserMedia()`.

```
"origin": "https://streamyard.com",
"request_type": "getUserMedia",
"video": {
  "width": {"ideal": 854},
  "height": {"ideal": 480},
```

```
"frameRate": {"max": 30, "ideal": 30}
}
```

O trecho acima mostra que a aplicação solicitou vídeo com resolução ideal de 854 × 480 pixels e taxa de quadros ideal de 30 FPS.

No lado da Júlia, o dispositivo identificado foi:

```
"label": "ACER HD User Facing (04f2:b773)"
```

No lado do Wagner, o dispositivo identificado foi:

```
"label": "Integrated Camera (5986:0397)"
```

4.2. Áudio

A direção do fluxo de mídia é indicada na própria descrição SDP por meio do atributo `sendonly`, presente tanto na linha principal da mídia quanto como atributo explícito, caracterizando um fluxo unidirecional de áudio, no qual o emissor apenas envia mídia. Os codecs ofertados para essa áudio são apresentados na linha `m=audio`, onde são listados os identificadores de payload RTP suportados, e detalhados nos atributos `a=rtpmap` e `a=fmtp`, que especificam, respectivamente, o tipo de codec, a taxa de amostragem, o número de canais e os parâmetros adicionais de operação.

```
▼ m=audio 9 UDP/TLS/RTP/SAVPF 111 63 9 0 8 13 110 126 (29 more lines) direction=sendonly mid=0
c=IN IP4 0.0.0.0
a=rtcp:9 IN IP4 0.0.0.0
a=ice-ufrag:+5ZV
a=ice-pwd:5BwJyFAeFnPqxIBs1UvwiD40
a=ice-options:trickle
a=fingerprint:sha-256 E0:2E:4C:A9:B7:79:60:AA:37:70:B4:FE:C1:F1:AA:49:05:45:9D:F0:31:68:42:CF:E6:6E:2F:3D:27:49:BD:
a=setup:actpass
a=mid:0
a=extmap:1 urn:ietf:params:rtp-hdext:ssrc-audio-level
a=extmap:2 http://www.webrtc.org/experiments/rtp-hdext/abs-send-time
a=extmap:3 http://www.ietf.org/id/draft-holmer-rmcat-transport-wide-cc-extensions-01
a=extmap:4 urn:ietf:params:rtp-hdext:sdes:mid
a=sendonly
a=msid:43e187b7-f4df-4897-85c8-cf31edd3aca2 02597cb9-773a-4c68-a650-4d0497403bfe
a=rtcp-mux
a=rtcp-rsize
a=rtpmap:111 opus/48000/2
a=rtcp-fb:111 transport-cc
a=fmtp:111 minptime=10;useinbandfec=1
a=rtpmap:63 red/48000/2
a=fmtp:63 111/111
a=rtpmap:9 G722/8000
a=rtpmap:0 PCMU/8000
a=rtpmap:8 PCMA/8000
a=rtpmap:13 CN/8000
a=rtpmap:110 telephone-event/48000
a=rtpmap:126 telephone-event/8000
a:ssrc:894125422 cname:tZvPu+BpldP+NW09
a:ssrc:894125422 msid:43e187b7-f4df-4897-85c8-cf31edd3aca2 02597cb9-773a-4c68-a650-4d0497403bfe
```

Figure 8: Negociação de áudio.

Payload	Codec	Taxa	Canais	Observação
111	Opus	48 kHz	2	Codec principal
63	RED	48 kHz	2	Redundância
9	G.722	8 kHz	1	Banda larga
0	PCMU	8 kHz	1	G.711 μ -law
8	PCMA	8 kHz	1	G.711 A-law
13	CN	8 kHz	—	Comfort Noise
110	telephone-event	48 kHz	—	DTMF
126	telephone-event	8 kHz	—	DTMF

O codec de áudio efetivamente utilizado foi o Opus. As estatísticas extraídas do WebRTC Internals indicaram payload type 111, frequência de amostragem de 48 kHz e dois canais.

```
▼ m=audio 9 UDP/TLS/RTP/SAVPF 111 (16 more lines) direction=recvonly mid=0
c=IN IP4 35.193.30.155
a=recvonly
a=mid:0
a=rtcp-mux
a=ice-ufrag:2nSQ
a=ice-pwd:/30tFZse2t30l4+YckIHDu
a=ice-options:trickle
a=setup:active
a=rtpmap:111 opus/48000/2
a=fmtp:111 useinbandfec=1;sprop-stereo=0;stereo=0;
a=extmap:4 urn:ietf:params:rtp-hdrext:sdes:mid
a=msid:janus janus0
a=ssrc:1296513097 cname:janus
a=candidate:1 1 udp 2015363327 35.193.30.155 51677 typ host
a=candidate:2 1 udp 2015363583 35.193.30.155 57118 typ host
a=end-of-candidates
```

Figure 9: Escolha do codec de áudio.

```
"mimeType": "audio/opus",
"clockRate": 48000,
"channels": 2,
"payloadType": 111
```

Também foi observado o uso de FEC em banda, indicado pelo parâmetro `useinbandfec=1`.

```
"codec": "opus (111, ;sprop-stereo=0;stereo=0;useinbandfec=1)"
```

A escolha do Opus é adequada para comunicação em tempo real, pois esse codec oferece boa qualidade de áudio e adaptação eficiente a variações de rede.

Parâmetro	Valor observado
Codec de áudio	Opus
Payload Type	111
Frequência	48 kHz
Canais	2
FEC	Habilitado

4.3. Vídeo

Abaixo estão alguns dos codecs de vídeo ofertados na SDP, conforme listados na linha `m=video` e detalhados pelos atributos `a=rtpmap` e `a=fmtp`. Observa-se a oferta de múltiplos codecs modernos, como VP8, VP9, H.264, H.265 e AV1, além de mecanismos auxiliares de retransmissão (RTX), redundância (RED) e correção de erros (ULPFEC).

```

▼ m=video 9 UDP/TLS/RTP/SAVPF 96 97 103 104 107 108 109 114 115 116 117 118 39 40 45 46 98 99 100 101 119 120 49 50 123 124 125 (139 mo
c=IN IP4 0.0.0.0
a=rtcp:9 IN IP4 0.0.0.0
a=ice-ufrag:+5ZV
a=ice-pwd:5BwJyFAeFnPqxIBs1UvwiD40
a=ice-options:trickle
a=fingerprint:sha-256 E0:2E:4C:A9:B7:79:60:AA:37:70:B4:FE:C1:F1:AA:49:05:45:9D:F0:31:68:42:CF:E6:6E:2F:3D:27:49:BD:03
a=setup:actpass
a=mid:1
a=extmap:14 urn:ietf:params:rtp-hdrex:toffset
a=extmap:2 http://www.webrtc.org/experiments/rtp-hdrex/abs-send-time
a=extmap:13 urn:3gpp:video-orientation
a=extmap:3 http://www.ietf.org/id/draft-holmer-rmcat-transport-wide-cc-extensions-01
a=extmap:5 http://www.webrtc.org/experiments/rtp-hdrex/playout-delay
a=extmap:6 http://www.webrtc.org/experiments/rtp-hdrex/video-content-type
a=extmap:7 http://www.webrtc.org/experiments/rtp-hdrex/video-timing
a=extmap:8 http://www.webrtc.org/experiments/rtp-hdrex/color-space
a=extmap:4 urn:ietf:params:rtp-hdrex:sdes:mid
a=extmap:10 urn:ietf:params:rtp-hdrex:sdes:rtp-stream-id
a=extmap:11 urn:ietf:params:rtp-hdrex:sdes:repaired-rtp-stream-id
a=sendonly
a=msid:43e187b7-f4df-4897-85c8-cf31edd3aca2 56607da3-6e4a-49a2-9f1d-0f42845fab07
a=rtcp-mux
a=rtcp-rsize
a=rtpmap:96 VP8/90000
a=rtcp-fb:96 goog-remb
a=rtcp-fb:96 transport-cc
a=rtcp-fb:96 ccm fir
a=rtcp-fb:96 nack
a=rtcp-fb:96 nack pli
a=rtpmap:97 rtx/90000
a=fmtp:97 apt=96
a=rtpmap:103 H264/90000
a=rtcp-fb:103 goog-remb
a=rtcp-fb:103 transport-cc
a=rtcp-fb:103 ccm fir
a=rtcp-fb:103 nack
a=rtcp-fb:103 nack pli
a=fmtp:103 level-asymmetry-allowed=1;packetization-mode=1;profile-level-id=42001f
a=rtpmap:104 rtx/90000
a=fmtp:104 apt=103
a=rtpmap:107 H264/90000
a=rtcp-fb:107 goog-remb
a=rtcp-fb:107 transport-cc
a=rtcp-fb:107 ccm fir
a=rtcp-fb:107 nack
a=rtcp-fb:107 nack pli
a=fmtp:107 level-asymmetry-allowed=1;packetization-mode=0;profile-level-id=42001f
a=rtpmap:108 rtx/90000
a=fmtp:108 apt=107
a=rtpmap:109 H264/90000
a=rtcp-fb:109 goog-remb
a=rtcp-fb:109 transport-cc
a=rtcp-fb:109 ccm fir
a=rtcp-fb:109 nack
a=rtcp-fb:109 nack pli
a=fmtp:109 level-asymmetry-allowed=1;packetization-mode=1;profile-level-id=42e01f
a=rtmap:114 rtx/90000

```

Figure 10: Início da negociação de vídeo.

Codec	Payloads	Taxa	Variantes / Observações
VP8	96	90 kHz	Codec base
VP9	98, 100	90 kHz	Perfis 0 e 2
H.264	103, 107, 109, 115, 117, 39, 119	90 kHz	Múltiplos perfis e packetization-modes
AV1	45	90 kHz	Codec avançado
H.265 (HEVC)	49	90 kHz	Alta eficiência

Para o vídeo, o codec efetivamente utilizado foi o VP8, identificado pelo payload type 96 e clock rate de 90 kHz.

```
▼ m=video 9 UDP/TLS/RTP/SAVPF 96 97 (26 more lines) direction=recvonly mid=1
c=IN IP4 35.193.30.155
a=recvonly
a=mid:1
a=rtcp-mux
a=ice-ufrag:2nSQ
a=ice-pwd:/30tFZse2t30l4+YckIHdu
a=ice-options:trickle
a=setup:active
a=rtpmap:96 VP8/90000
a=rtcp-fb:96 ccm fir
a=rtcp-fb:96 nack
a=rtcp-fb:96 nack pli
a=rtcp-fb:96 goog-remb
a=rtcp-fb:96 transport-cc
a=extmap:13 urn:3gpp:video-orientation
a=extmap:3 http://www.ietf.org/id/draft-holmer-rmcat-transport-wide-cc-extensions-01
a=extmap:4 urn:ietf:params:rtp-hdrext:sdes:mid
a=extmap:10 urn:ietf:params:rtp-hdrext:sdes:rtp-stream-id
a=extmap:11 urn:ietf:params:rtp-hdrext:sdes:repaired-rtp-stream-id
a=rtpmap:97 rtx/90000
a=fmtp:97 apt=96
a=msid:janus janus1
a=ssrc:3919611639 cname:janus
a=candidate:1 1 udp 2015363327 35.193.30.155 51677 typ host
a=candidate:2 1 udp 2015363583 35.193.30.155 57118 typ host
a=end-of-candidates
```

Figure 11: Escolha do codec de vídeo.

```
"mimeType": "video/VP8",
"clockRate": 90000,
"payloadType": 96
```

Nas estatísticas RTP de saída, o WebRTC Internals também registrou o uso do codec VP8:

```
"codec": "VP8 (96)"
```

Embora a captura inicial tenha solicitado 854×480 pixels, a resolução efetivamente transmitida foi reduzida durante a sessão.

```
"frameWidth": [427, 427, 427, 427],
"frameHeight": [240, 240, 240, 240]
```

A taxa de quadros também variou ao longo da transmissão:

```
"framesPerSecond": [23, 22, 18, 17]
```

Esses dados indicam que o WebRTC realizou adaptação dinâmica da qualidade da mídia, reduzindo resolução e taxa de quadros conforme as condições da sessão.

Parâmetro	Valor observado
Codec de vídeo	VP8
Payload Type	96
Clock Rate	90 kHz
Resolução efetiva	427 × 240
Taxa de quadros	17 a 23 FPS

5. Escolha de Caminho

A escolha do caminho de comunicação foi realizada utilizando ICE. Esse mecanismo é responsável por testar diferentes candidatos de conectividade e selecionar o melhor caminho disponível entre os participantes.

O dump do WebRTC Internals mostrou a configuração dos servidores STUN e TURN utilizados pelo StreamYard:

```
"iceServers": [  
  {  
    "urls": ["stun:turn-prod-1.streamyard.com:443"]  
  },  
  {  
    "urls": [  
      "turn:turn-prod-1.streamyard.com:443?transport=udp",  
      "turn:turn-prod-1.streamyard.com:443?transport=tcp",  
      "turns:turn-prod-1.streamyard.com:443?transport=tcp"  
    ]  
  }  
]
```

O STUN é utilizado para descoberta de endereços públicos em redes com NAT. O TURN permite que a mídia seja encaminhada por um servidor intermediário quando a comunicação direta não é possível. O uso de turns indica a possibilidade de utilização de TURN sobre conexão segura.

Na captura local também foi identificado um candidato do tipo host, associado ao endereço privado da máquina:

```
"candidateType": "host",
"address": "192.168.15.4",
"protocol": "udp",
"networkType": "wifi"
```

Além disso, foi encontrado o candidato do tipo server reflexive (srflx), obtidos a partir de um servidor STUN da infraestrutura do StreamYard, que revelam o endereço público do dispositivo por trás de um NAT. Apesar da ocorrência de eventos onicecandidateerror, o processo de coleta foi concluído com sucesso, resultando na seleção de um par de candidatos com estado succeeded. O estado final do transporte ICE como connected confirma que a sessão de mídia foi estabelecida com êxito.

```
▼ onicecandidate
16/06/2026, 21:37:58 {"sdpMid":"0","sdpMLineIndex":0,"candidate":{"candidate:3403383510 1 udp 1685921535 177.189.193.245 49324 typ srflx
```

Figure 12: Tipos de candidatos.

Endereço IP	177.189.193.245
Porta	49324
Protocolo	UDP
Tipo de candidato	Server Reflexive (srflx)
Endereço local associado	192.168.15.4

O par de candidatos selecionado apresentou estado bem-sucedido:

```
"state": "succeeded",
"nominated": true,
"currentRoundTripTime": 0.16
```

Esse resultado indica que o ICE selecionou com sucesso um caminho válido para transporte da mídia, com RTT aproximado de 160 ms.

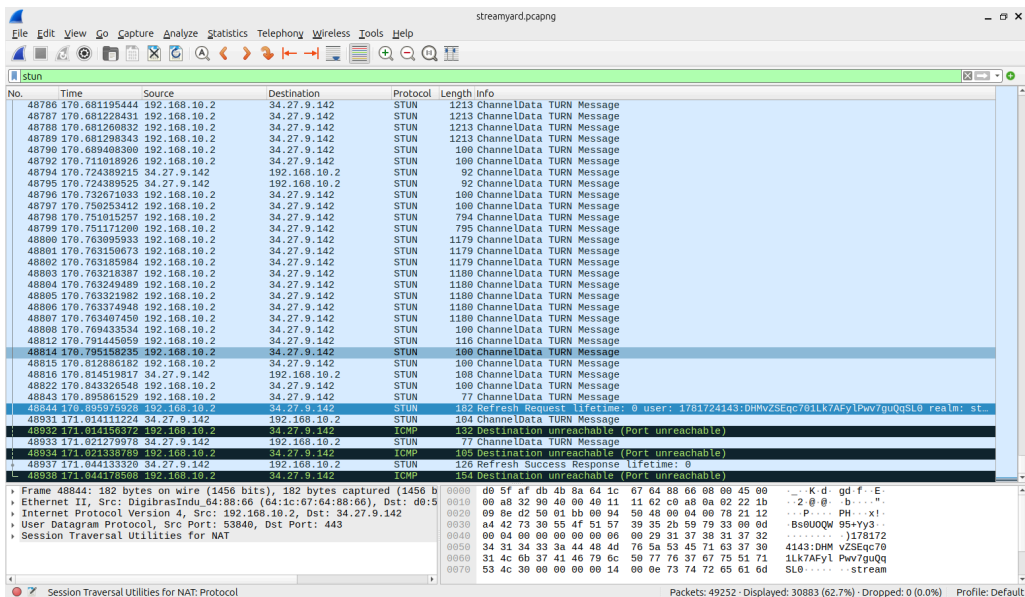


Figure 13: Captura no Wireshark com filtro STUN, evidenciando mensagens utilizadas na escolha de caminho.

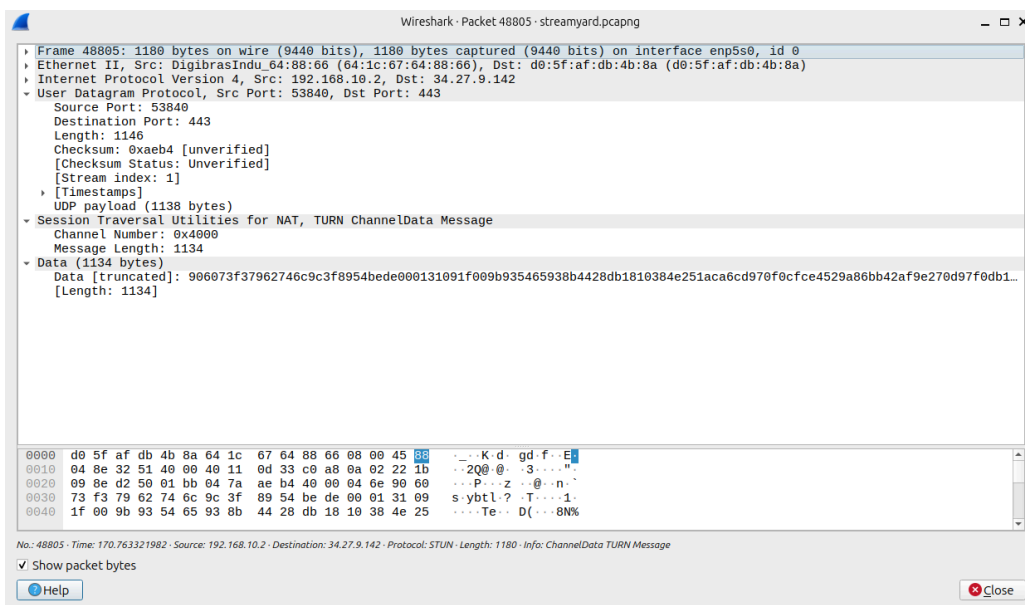


Figure 14: Mensagens TURN ChannelData observadas durante a transmissão.

6. Transporte de Mídia

Após o estabelecimento da sessão, o transporte da mídia ocorreu através dos protocolos definidos pelo WebRTC. O perfil utilizado foi:

UDP/TLS/RTP/SAVPF

Esse perfil indica o uso de UDP como protocolo de transporte, DTLS para segurança, RTP/SRTP para transporte da mídia e RTCP para controle e monitoramento.

A análise da captura de rede no Wireshark revelou predominância de tráfego UDP, característica comum em aplicações multimídia em tempo real.

Protocolo	Percentual observado
UDP	79,9%
STUN	62,7%
RTCP	0,8%
TLS	4,3%

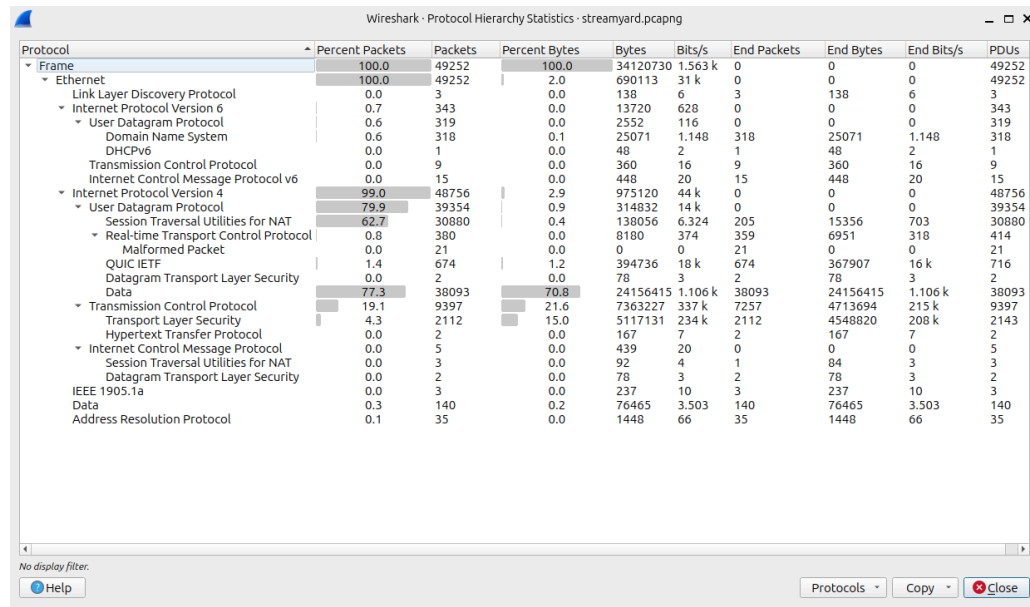


Figure 15: Hierarquia de protocolos capturada no Wireshark durante o experimento.

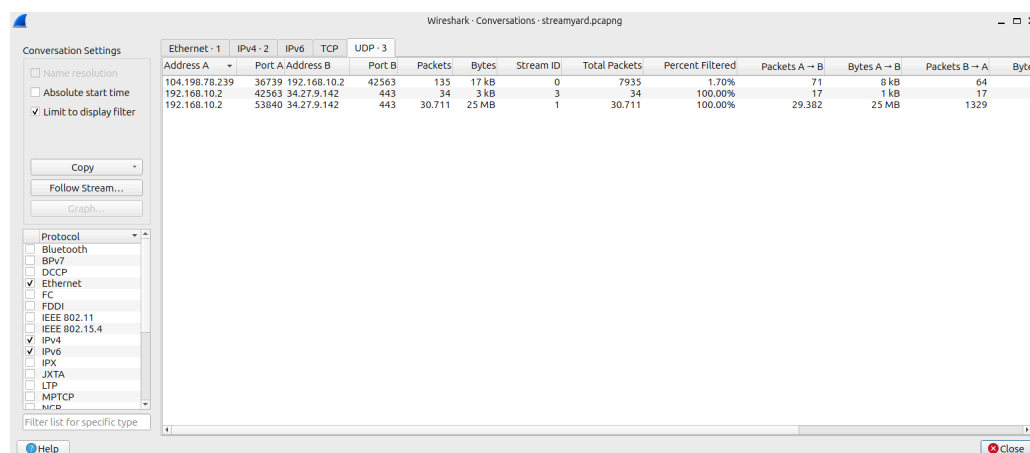


Figure 16: Conversações UDP observadas durante a sessão multimídia.

A análise das conversações UDP mostrou que o principal fluxo de dados ocorreu entre o endereço local 192.168.10.2 e o servidor 34.27.9.142, utilizando a porta 443. Foram observados aproximadamente 30 mil pacotes e cerca de 25 MB de dados transmitidos durante a sessão, evidenciando a utilização de um servidor TURN para encaminhamento da mídia.

O protocolo RTP é responsável pelo transporte dos fluxos de áudio e vídeo, enquanto o RTCP realiza o monitoramento da qualidade da transmissão, fornecendo estatísticas como perda de pacotes, jitter e atraso.

7. Estatísticas da Sessão

As estatísticas extraídas do WebRTC Internals permitiram avaliar o comportamento da sessão durante o experimento.

No fluxo de vídeo de saída, foram observados 2391 pacotes enviados no último ponto da amostragem analisada:

```
"packetsSent": [1409, 2346, 2364, 2391]
```

No fluxo de áudio de saída, foram observados 3505 pacotes enviados:

```
"packetsSent": [2083, 3419, 3454, 3505]
```

Não foram observadas retransmissões nos fluxos analisados:

```
"retransmittedPacketsSent": [0, 0, 0, 0]
```

Além disso, a perda de pacotes observada foi nula nos trechos analisados:

```
"packetsLost": [0, 0, 0, 0]
```

O jitter observado no áudio e no vídeo permaneceu reduzido:

```
"audio_jitter": [0.002020, 0.001270, 0.001562, 0.000812],
```

```
"video_jitter": [0.013111, 0.012988, 0.008199, 0.003244]
```

Assim, o jitter final observado foi de aproximadamente 0,8 ms para áudio e 3,2 ms para vídeo. Esses valores indicam estabilidade na transmissão durante o trecho analisado.

Indicador	Valor observado
RTT	aprox. 160 ms
Jitter de áudio	aprox. 0,8 ms
Jitter de vídeo	aprox. 3,2 ms
Perda de pacotes	0 pacotes
Retransmissões	0 pacotes

8. Qualidade de Serviço

O WebRTC não garante QoS de forma rígida como ocorre em redes com reserva explícita de recursos, porém utiliza mecanismos de adaptação e monitoramento para manter a qualidade da comunicação em tempo real.

Durante o experimento, foram observados os seguintes mecanismos relacionados à qualidade de serviço:

- monitoramento por RTCP;
- cálculo de RTT;
- medição de jitter;
- contagem de perda de pacotes;
- adaptação de resolução;
- adaptação da taxa de quadros;
- controle de bitrate.

A redução da resolução efetiva de 854×480 para 427×240 pixels e a variação da taxa de quadros entre 17 e 23 FPS indicam adaptação dinâmica da transmissão.

```
"qualityLimitationReason": ["none", "none", "none", "none"],  
"qualityLimitationResolutionChanges": [0, 0, 0, 0]
```

No trecho analisado, não foram indicadas limitações explícitas de qualidade pelo navegador, e também não ocorreram mudanças adicionais de resolução durante a amostragem registrada.

9. Segurança

A segurança da sessão WebRTC foi observada tanto no dump do WebRTC Internals quanto nas capturas do Wireshark.

As estatísticas de certificado indicaram o uso do algoritmo SHA-256 para fingerprint:

```
"fingerprintAlgorithm": ["sha-256", "sha-256", "sha-256", "sha-256"]
```

Esse mecanismo é utilizado para autenticação dos certificados DTLS empregados na sessão.

Além disso, o perfil UDP/TLS/RTP/SAVPF indica que a mídia foi transportada de forma segura, utilizando DTLS para negociação de chaves e SRTP para proteção dos fluxos RTP.

Foram identificados os seguintes mecanismos de segurança:

Protocolo	Função
DTLS	Negociação segura de chaves
SRTP	Proteção criptográfica da mídia
SHA-256	Fingerprint dos certificados
TLS	Proteção de conexões auxiliares da aplicação

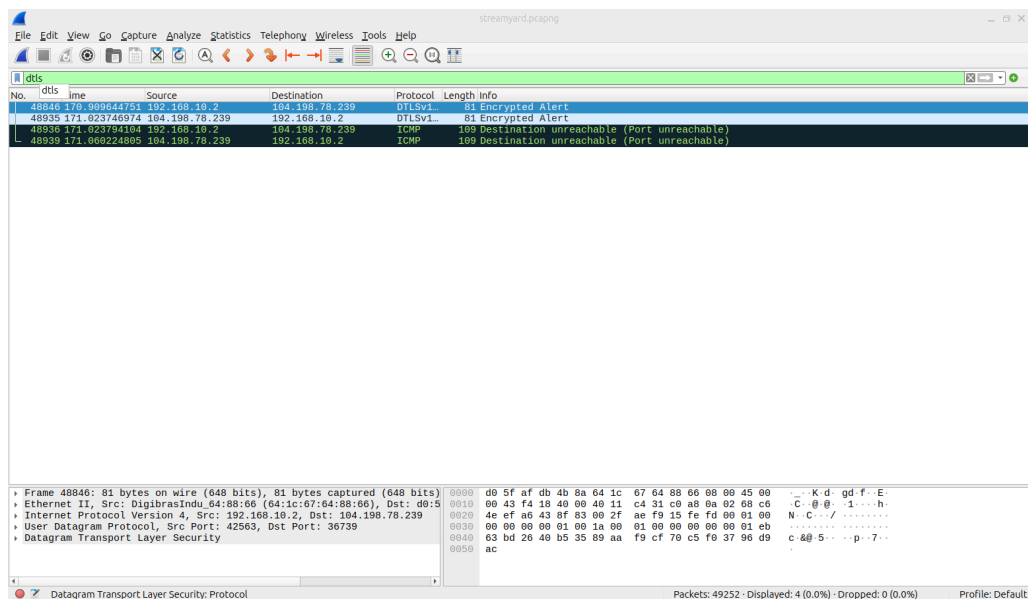


Figure 17: Pacotes DTLS observados na captura do Wireshark.

10. Conclusão

A análise realizada demonstrou o funcionamento de uma sessão multimídia em tempo real baseada em WebRTC utilizando a plataforma StreamYard.

Foram observados os mecanismos de sinalização responsáveis pela negociação da sessão, a captura de áudio e vídeo através da API `getUserMedia()`, a utilização dos codecs Opus e VP8, os protocolos ICE, STUN e TURN empregados para escolha de caminho e travessia de NAT, além do transporte de mídia baseado em UDP, RTP, RTCP e SRTP.

As estatísticas coletadas indicaram RTT aproximado de 160 ms, jitter reduzido e ausência de perda de pacotes nos trechos analisados. Também foi possível observar adaptação dinâmica de resolução e taxa de quadros, característica importante em aplicações WebRTC.

Por fim, as capturas confirmaram o uso de mecanismos de segurança como DTLS, SRTP e fingerprints SHA-256, demonstrando que a comunicação audiovisual foi realizada de forma protegida.

11. Referências

- STREAMYARD. Plataforma de transmissão ao vivo. Disponível em: <https://streamyard.com>. Acesso em: 16 jun. 2026.
- IETF. RFC 8445 – Interactive Connectivity Establishment (ICE).
- IETF. RFC 5389 – Session Traversal Utilities for NAT (STUN).
- IETF. RFC 5766 – Traversal Using Relays around NAT (TURN).
- IETF. RFC 3550 – RTP: A Transport Protocol for Real-Time Applications.
- IETF. RFC 5764 – Datagram Transport Layer Security (DTLS) Extension to SRTP.
- IETF. RFC 8839 – Session Description Protocol (SDP) Offer/Answer for WebRTC.
- Google. WebRTC Internals Documentation. Disponível em: <https://webrtc.org>.